

# Unidad 6

---

## Creación de documentación para aplicaciones

---

DESARROLLO DE INTERFACES. 2º DAM.



---

# 1. Documentación

---

UD6. CREACIÓN DE DOCUMENTACIÓN PARA APLICACIONES

# 1. ¿Por qué es importante documentar?

---

- Facilitar la comprensión del sistema software y posibilitar su uso por parte del usuario.
- Facilitar el mantenimiento de la aplicación.
- Facilitar el desarrollo de mejoras por otros equipos de desarrollo.
- Facilitar que el sistema pueda ser comprendido por otros desarrolladores.
- Documentar es tan importante como el desarrollo de la aplicación.

## 2. Tipos de documentación

---

- Se pueden distinguir dos grandes grupos:
  - Documentación de pruebas.
  - Documentación técnica.



## 2.1. Documentación de pruebas

---

- **Documentación de entrada:** en la que se especifican los escenarios de prueba y se detallan los procedimientos de las mismas.
- **Documentación de salida:**
  - Informes resultantes de aplicar las pruebas indicadas en la *documentación de entrada*.
  - Recogen el histórico de pruebas realizadas, registrando así cada problema ocurrido y su corrección.

## 2.2. Documentación técnica

---

- Guías, hojas de especificaciones, manuales, etc...
- Se pueden distinguir dos grupos:
  - Documentación interna:
    - Comentarios en el código.
    - Incluir comentarios al comienzo de módulos.
    - Comentar variables, constantes, procedimientos y funciones.
    - No comentar lo obvio, ya que el exceso de comentarios puede ser contraproducente → El código debe ser lo suficientemente claro y estar ordenado.
  - Documentación externa: manual técnico (para desarrolladores) y/o guía de usuario (para facilitar su uso).



## 2.2.1.Documentación interna: Comentarios XML y APIs (Dependency Properties)

---

- Para las librerías DLL, la documentación interna no es solo para leer el código, sino para alimentar al **IntelliSense** de Visual Studio.
  - **Etiquetas XML:** Se usan comentarios de triple barra `///`. Lo que esté dentro, aparecerá luego en un cuadro flotante mientras escribe código.
  - **Etiqueta `<summary>`:** Describe qué hace la propiedad o el control.
  - **Etiqueta `<value>`:** Explica qué tipo de datos espera y su rango (útil en **Dependency Properties**).
  - **Aplicación en DPs:** Se debe documentar la propiedad *wrapper* de la Dependency Property.

## 2.2.1.Documentación interna: Comentarios XML y APIs (Dependency Properties)

- **Ejemplo:**

```
/// <summary>  
/// Define el porcentaje de carga del visor (0 a 100).  
/// </summary>  
/// <value>Acepta valores de tipo double.</value>  
public double ProgresoValor { ... }
```

- **Beneficio Real: IntelliSense**

- Al usar la DLL en otro proyecto, Visual Studio lee estos comentarios y los muestra en una ventana flotante de ayuda mientras escribes código.
- **Resultado:** El programador sabe qué hace tu propiedad sin salir del editor.



## 2.2.2.Documentación externa: el formato Markdown (.md)

---

- **¿Qué es Markdown?** Un lenguaje de marcado ligero que permite dar formato a un texto usando símbolos sencillos.
- **¿Por qué se usa?** Es el estándar en GitHub y entornos profesionales. Es fácil de leer tanto en crudo como renderizado.
- Sintaxis básica:
  - # Título, ## Subtítulo
  - **\*\*Negrita\*\***, *\*Cursiva\**
  - `Código en línea` y bloques de código con ```.
  - ![Texto](url\_imagen) para capturas de pantalla, etc.

NOTA: Ideal para el archivo **README.md** que acompaña a una librería DLL, explicando cómo instalarla y qué propiedades tiene.

## 2.3. Manuales y guías

- Diferentes tipos según el contenido y manera de utilización.
- Diferentes formatos y soportes: papel, web de ayuda, presentación, vídeo o una combinación de ellos.



## 2.3.1. Manual y guía de usuario

---

- Contiene toda la información para hacer más fácil el uso y la comprensión del programa.
- Ámbitos de aplicación:
  - Formación del usuario.
  - Guía de consulta.
  - Ayuda para detectar y corregir errores.
  - Etc.

## 2.3.1. Manual y guía de usuario

---

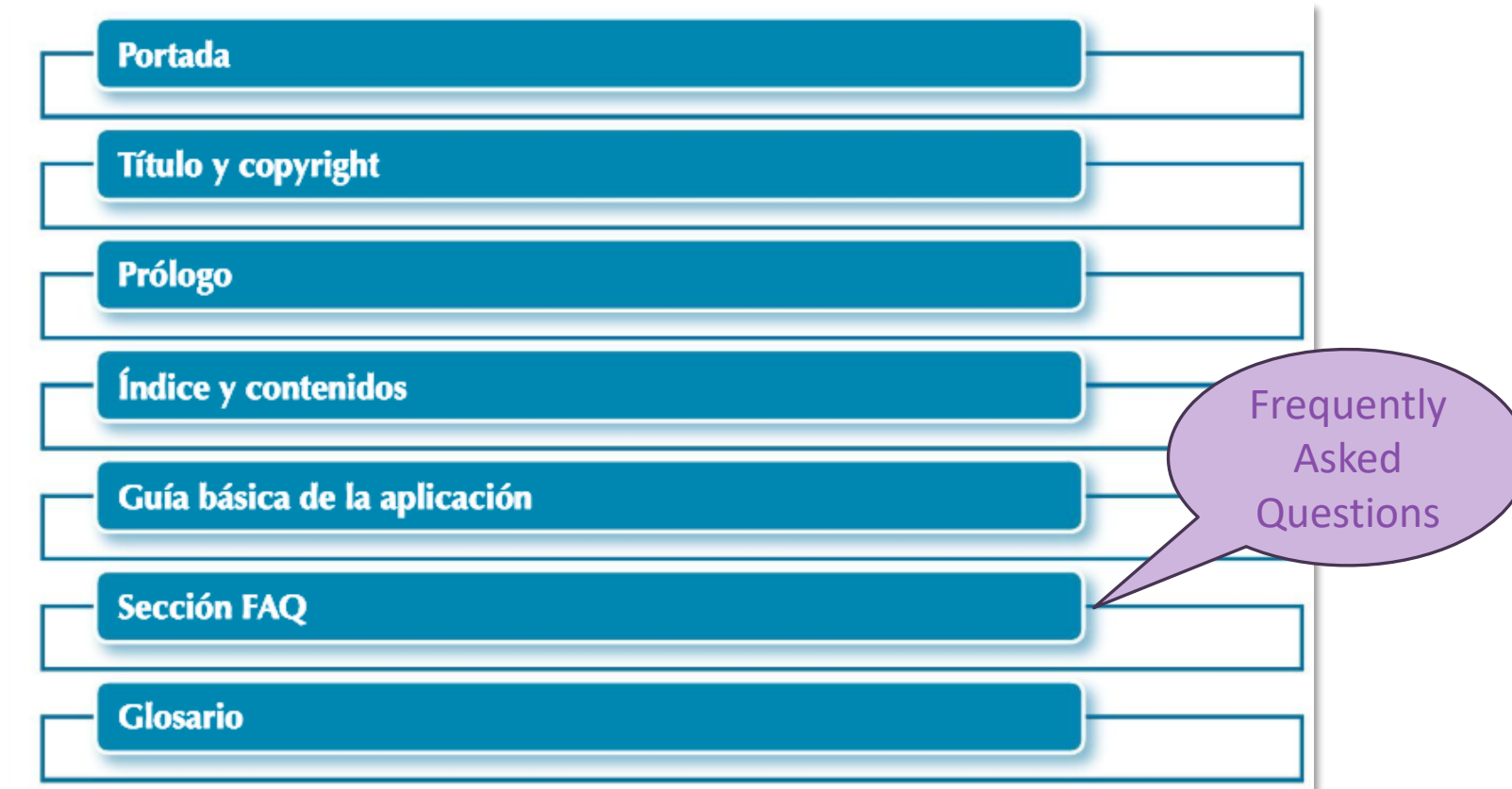
- El documento debe ser claro y atractivo.
- Se recomienda utilizar imágenes y gráficos.
- Emplear un vocabulario claro y conciso.
- Incluir ejemplos prácticos.



## 2.3.1. Manual y guía de usuario

---

- Posible estructura de manual de usuario de una aplicación:



## 2.3.2. Manual y guía de explotación

---

- Información relacionada con:
  - Instalación.
  - Configuración.
  - Puesta en marcha.
- Debe incluir:
  - Requerimientos mínimos del sistema.
  - Proceso de instalación.
  - Actualizaciones del sistema y copias de seguridad.
  - Glosario.
  - Índice.
- Se puede dividir en *manual de instalación* y *manual de configuración* en sistemas complejos.

## 2.3.3. Guía rápida y guía de referencia

---

- Guía rápida:
  - Orientada a usuarios del sistema y/o encargados del mantenimiento.
  - Aportan **información** muy **concreta** y muy diferente, relacionada con diversos procedimientos o campos de una aplicación.
- Guía de referencia:
  - Orienta a usuarios que tienen un mayor nivel de conocimiento sobre el uso de la aplicación y experiencia.
  - Contienen información más técnica:
    - Mensajes de error y su causa.
    - Tipos de datos de entrada permitidos.
    - Tipos de comandos.
    - Etc.

## 2.4. Ficheros de ayuda

---

- Documento/archivo que contiene información de ayuda sobre un determinado campo.
  - Su objetivo es guiar a los usuarios en tareas específicas, resolver dudas, y proporcionar referencias rápidas sobre las funcionalidades o problemas comunes. Su objetivo es ofrecer una **ayuda interactiva y organizada**. A diferencia de un documento PDF largo que se lee de arriba abajo, un fichero de ayuda está pensado para que el usuario **encuentre la solución a un problema concreto en segundos**.

## 2.4. Ficheros de ayuda

---

- **Características clave:**

Un buen fichero de ayuda tiene tres "patas":

- **Navegación:** Tiene un índice (tabla de contenidos) y un buscador por palabras clave.
- **Contexto:** Si el usuario está en la ventana de, por ejemplo, "Añadir Animal", el fichero de ayuda debería poder abrirse directamente en la sección que explica esa ventana.
- **Interconectividad:** Contiene hipervínculos (enlaces) que te llevan de un tema a otro relacionado.

## 2.4.1. Formatos

---

- **CHM (Microsoft Compiled HTML Help)**
  - Contiene una tabla de contenidos, índice, y buscador integrado.
  - Permite navegación jerárquica y vista de información contextual. diseñado específicamente para **documentación o ayuda integrada** en aplicaciones de escritorio.
  - **Usos:** Documentación técnica estructurada, como manuales y ayudas.
  - **Ventajas:** Ideal para proyectos grandes. Soporta glosarios y tablas de contenido detalladas.
- **HTML (Web Help o Documentación Web)**
  - Basado en navegadores web, soporta tablas de contenidos, glosarios, y búsqueda avanzada.
  - Permite personalización mediante CSS y JavaScript.
  - **Usos:** Documentación en línea accesible desde cualquier dispositivo.
  - **Ventajas:** Multiplataforma. Fácil de actualizar y mantener.
- **DocBook**
  - Basado en XML, permite estructurar documentos en secciones jerárquicas. Se puede convertir a HTML, PDF o CHM.
  - **Usos:** Documentación técnica estructurada, como manuales y ayudas.
  - **Ventajas:** Ideal para proyectos grandes. Soporta glosarios y tablas de contenido detalladas.

## 2.4.1. Formatos

---

- **JavaHelp**
  - Diseñado específicamente para aplicaciones Java.
  - Incluye mapa del fichero (Tabla de contenidos e índices) y búsqueda.
  - **Usos:** ayuda integrada en aplicaciones desarrolladas en Java.
  - **Ventajas:** compatible con entornos multiplataforma. Se adapta a las necesidades de los desarrolladores Java.
- **Help Viewer (Apple Help)**
  - Sistema de ayuda nativo en macOS.
  - Proporciona una tabla de contenidos, índice, y búsqueda para navegación.
  - **Usos:** ayuda integrada en aplicaciones de macOS.
  - **Ventajas:** Integración perfecta con el sistema operativo. Experiencia de usuario consistente con el resto del sistema.

## 2.4.2. Herramientas para generar ficheros de ayuda

---

- **HelpNDoc:** Herramienta fácil de usar para crear **CHM, HTML, PDF, Word y EPUB**, con soporte para **hipervínculos, búsqueda y capturas de pantalla**.
- **MadCap Flare:** Solución profesional que permite crear **documentación multiplataforma** (CHM, HTML5, PDF, etc.) con **gestión de contenido, traducción y localización**.
- **RoboHelp (Adobe):** Plataforma avanzada para crear **documentación interactiva**, con **personalización, búsqueda avanzada, módulos reutilizables** y soporte para **múltiples formatos**.
- **WinCHM:** Herramienta sencilla para generar **ficheros CHM**, con **importación de documentos HTML/Word, búsqueda y personalización** de la interfaz.

## 2.4.2. Herramientas para generar ficheros de ayuda

---

- **JavaHelp**: sistema de creación de ficheros de ayuda y, además, es un formato de ficheros de ayuda. Se caracteriza por:
  - Utilizar Java para su implementación.
  - Utilizarse en aplicaciones Java.
  - Ser independiente de la plataforma.
  - Facilitar la realización de ayuda online.
- **Javadoc**: herramienta gratuita similar a HELPMAKER. Permite incluir enlaces externos, insertar imágenes o crear índices.
- **Doxygen**: Herramienta de documentación para varios lenguajes, incluidos **Java**, que genera documentación a partir de comentarios en el código fuente.

# Tarea 1: Plan de Pruebas y Control de Calidad (QA)

---

**Objetivo:** Elaborar la documentación de entrada y salida para verificar que el programa cumple con lo que se pidió en la UD3, forzando situaciones límite para ver cómo reacciona el software.

**Instrucciones:** Basándote en tu proyecto de la UD3, debes crear un documento **(en PDF)** que incluya:

## 1. Documentación de Entrada (Plan de Verificación):

- Diseña una tabla con **5 pruebas de verificación de requisitos**.
- **Ejemplos obligatorios:**
  1. Verificación de la relación 1:N
  2. Verificación del Control Personalizado (¿Cambia de color el botón al pulsar?).
  3. Verificación de la Barra de Progreso (¿Recibe un valor aleatorio?).

# Tarea 1: Plan de Pruebas y Control de Calidad (QA)

---

## 2. Documentación de Salida (Informe de resultados):

- Ejecuta las pruebas diseñadas en tu aplicación.
- Registra si el resultado fue satisfactorio o si encontraste algún fallo (bug).
- En caso de que el programa se cierre o no haga nada, se debe documentar qué entrada causó el problema y qué crees que debería mejorar en el código para que no pasara.

## 3. Capturas de evidencia:

Adjunta imágenes que demuestren que has realizado las pruebas.

NOTA: se tendrá en cuenta para la calificación la calidad de la documentación: formato, estructura, etc.



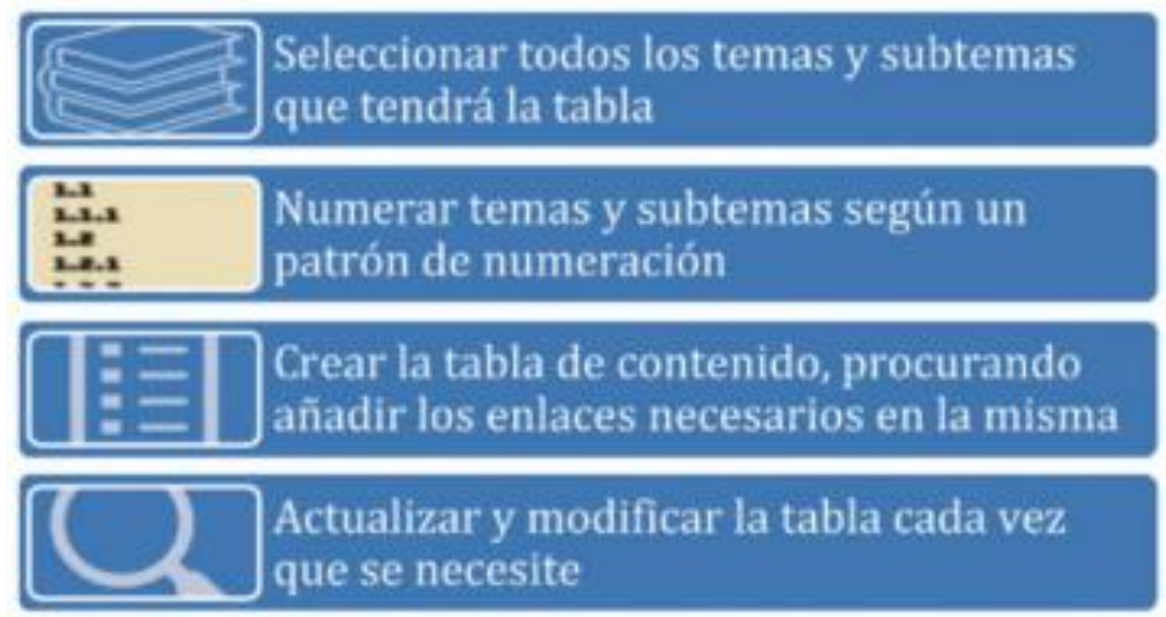
## 2.4.3. Ayuda genérica y sensible al contexto

---

- Ayuda genérica:
  - Contiene toda la información de ayuda sobre una determinada aplicación, dividida por temas o campos.
  - Presenta un índice o menú (generalmente).
  - Permite navegar o buscar entre los elementos que la forman.
- Ayuda sensible al contexto:
  - Se trata de **ayuda particularizada para un tema o acción concreta**.
  - Puede presentarse como ayuda en línea o breve detalle del uso de un menú en el programa.

## 2.4.4. Tablas de contenidos

- Nos ayudan a modelar tanto el contenido como la estructura de un documento determinado.
- La información se organiza en forma de esquema, con niveles (títulos y subtítulos).
- Contienen enlaces directos a cada título o subtítulo
- Suelen situarse al comienzo de cada documento.



# Tarea 2: Elaboración del Manual y Guía de Usuario



Crea una pequeña guía de usuario de la aplicación creada en la actividad 14 de la UD3.

Deberás hacer esta guía en un procesador de textos de forma que puedas incluir un índice de navegación. Su **extensión mínima será de 6 páginas** y máxima de 9. **Entrégala en pdf.**

## Estructura de la Tarea:

- ❖ **Portada:** Con el nombre de la app y sus datos.
- ❖ **Prólogo/Introducción:** ¿Qué hace la aplicación?
- ❖ **Índice de navegación:** Debe ser clicable en el PDF final.
- ❖ **Contenidos:** Navegación por la interfaz (aspecto), uso de las funcionalidades, preguntas frecuentes, soporte/ayuda. → deberán añadirse imágenes que ayuden al entendimiento de la guía.
- ❖ **Glosario.**

# Tarea 2: Consejos para la Guía de Usuario

---

- **Claridad y Atractivo:** el manual debe ser fácil de leer. Debéis usar un vocabulario conciso y evitar tecnicismos excesivos que el usuario final no entienda.
- **Uso Estratégico de Imágenes:** Tal como dice tu tarea, las imágenes son obligatorias. Recuerda que una captura de pantalla con flechas o números es mucho más útil que tres párrafos de texto explicando dónde está un botón.
- **La sección de FAQ:** intenta que las "Preguntas Frecuentes" incluyan lo que descubristeis en la Tarea 1 de pruebas. Por ejemplo: *"¿Por qué no puedo guardar? Asegúrese de que no hay campos vacíos"*



## 2.5. Documentación de la aplicación: Doxygen

---

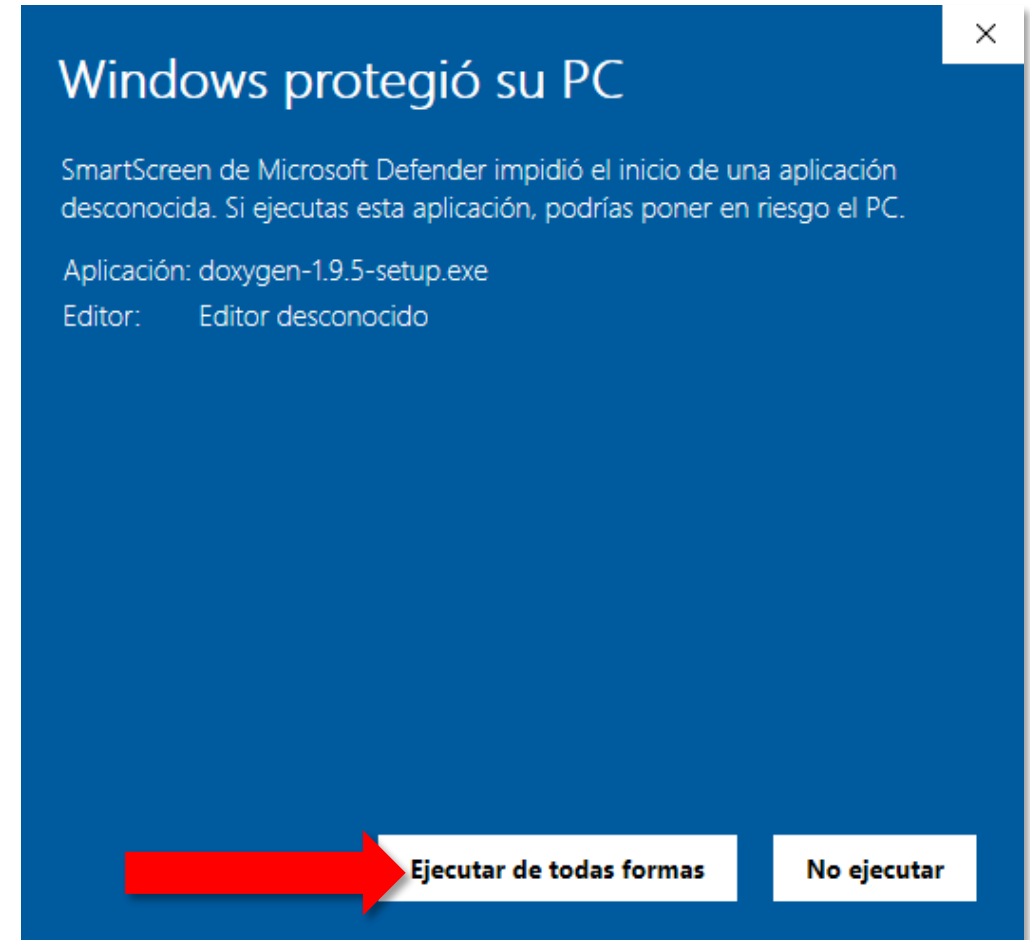
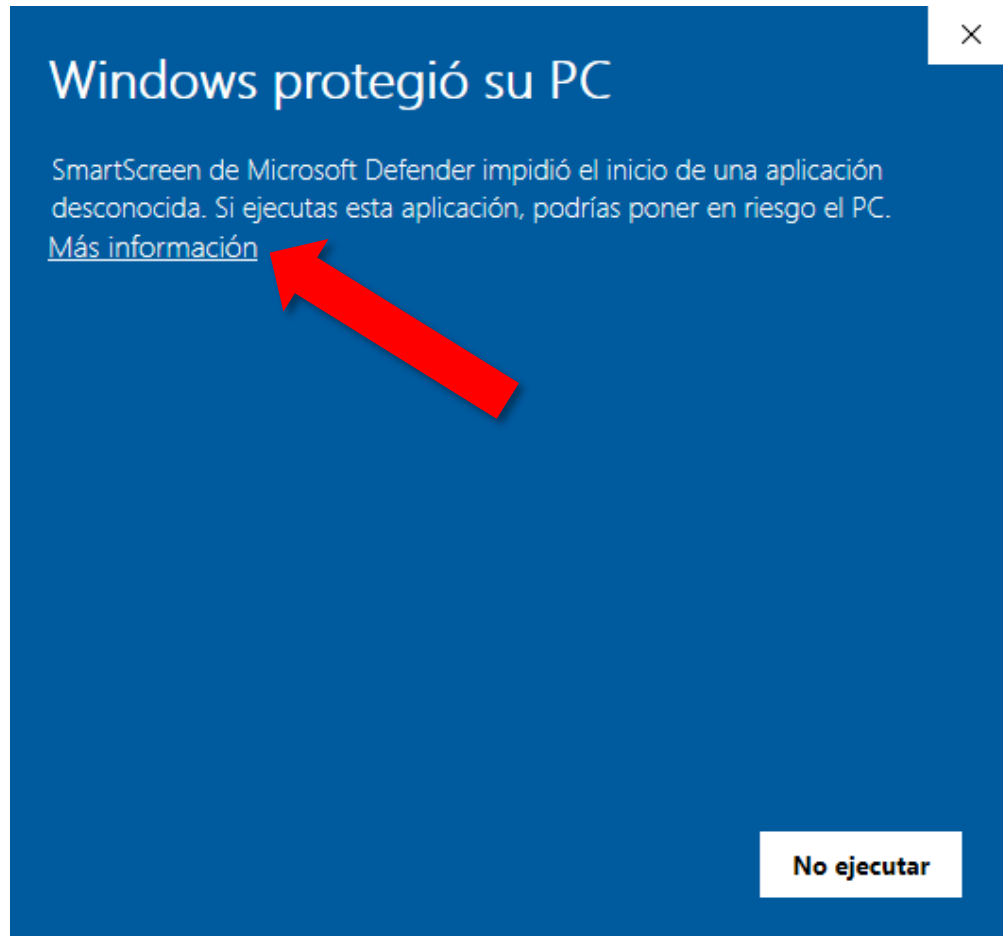
- Permite generar la documentación a partir del código fuente.
- Compatible con multitud de lenguajes: C/C++, Objective-C, C#, PHP, Java, Python, entre otros.
- Permite generar un navegador de documentación en línea (en HTML) y/o un manual de referencia fuera de línea.
- Permite visualizar las relaciones entre los elementos mediante gráficos de dependencia y diagramas de herencia que se generan automáticamente.
- Permite crear un manual de usuario normal.

## 2.5.1. Descarga e instalación de Doxygen

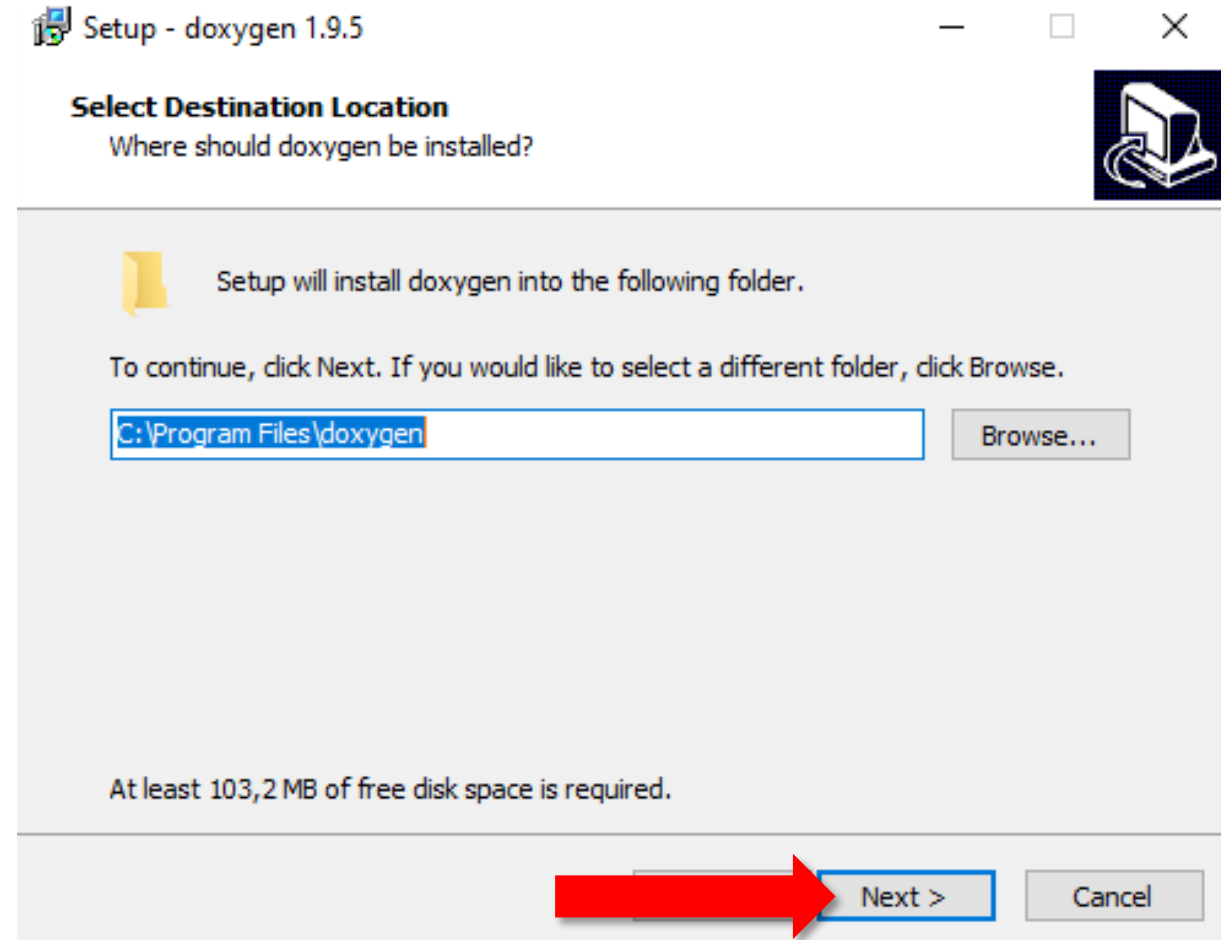
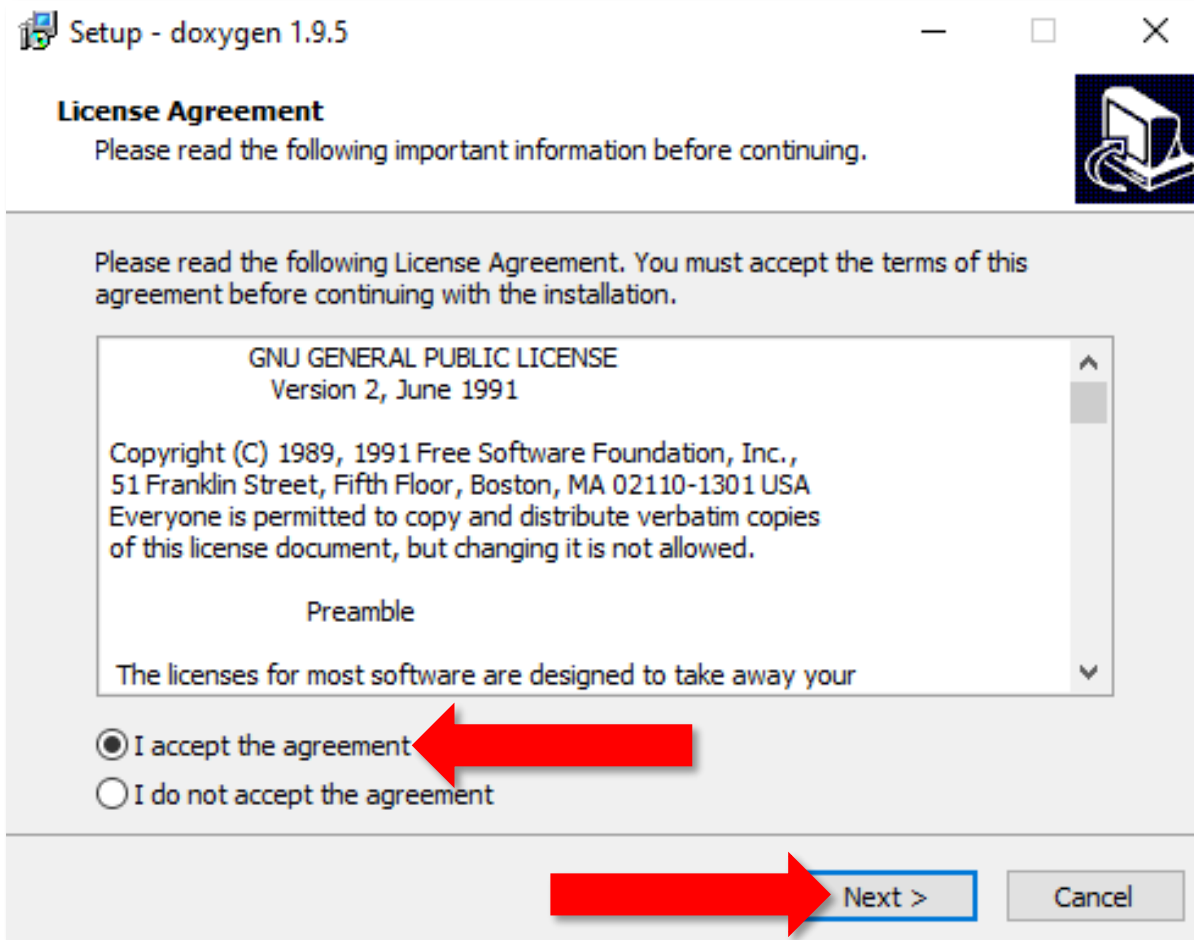
- Página de descarga: <https://www.doxygen.nl/download.html>



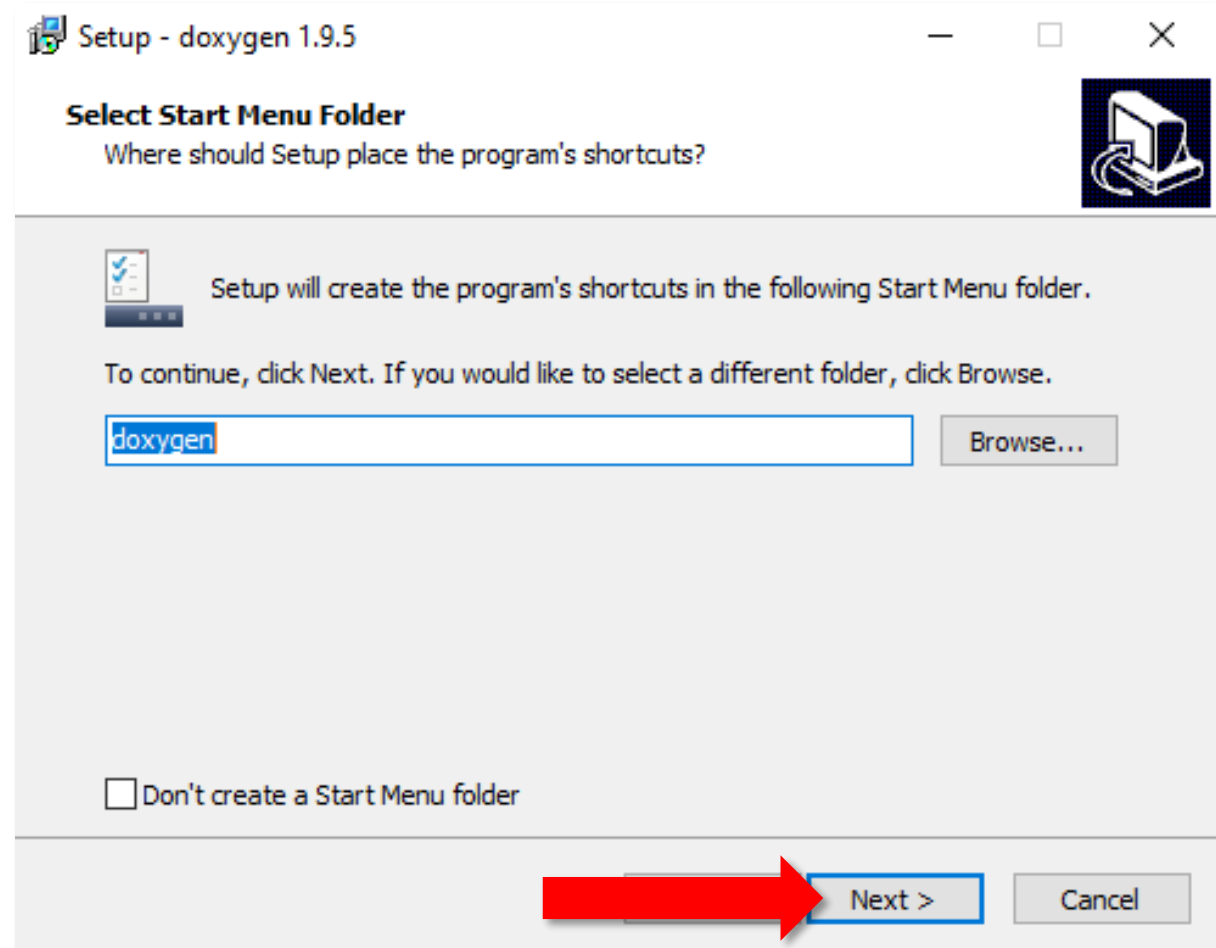
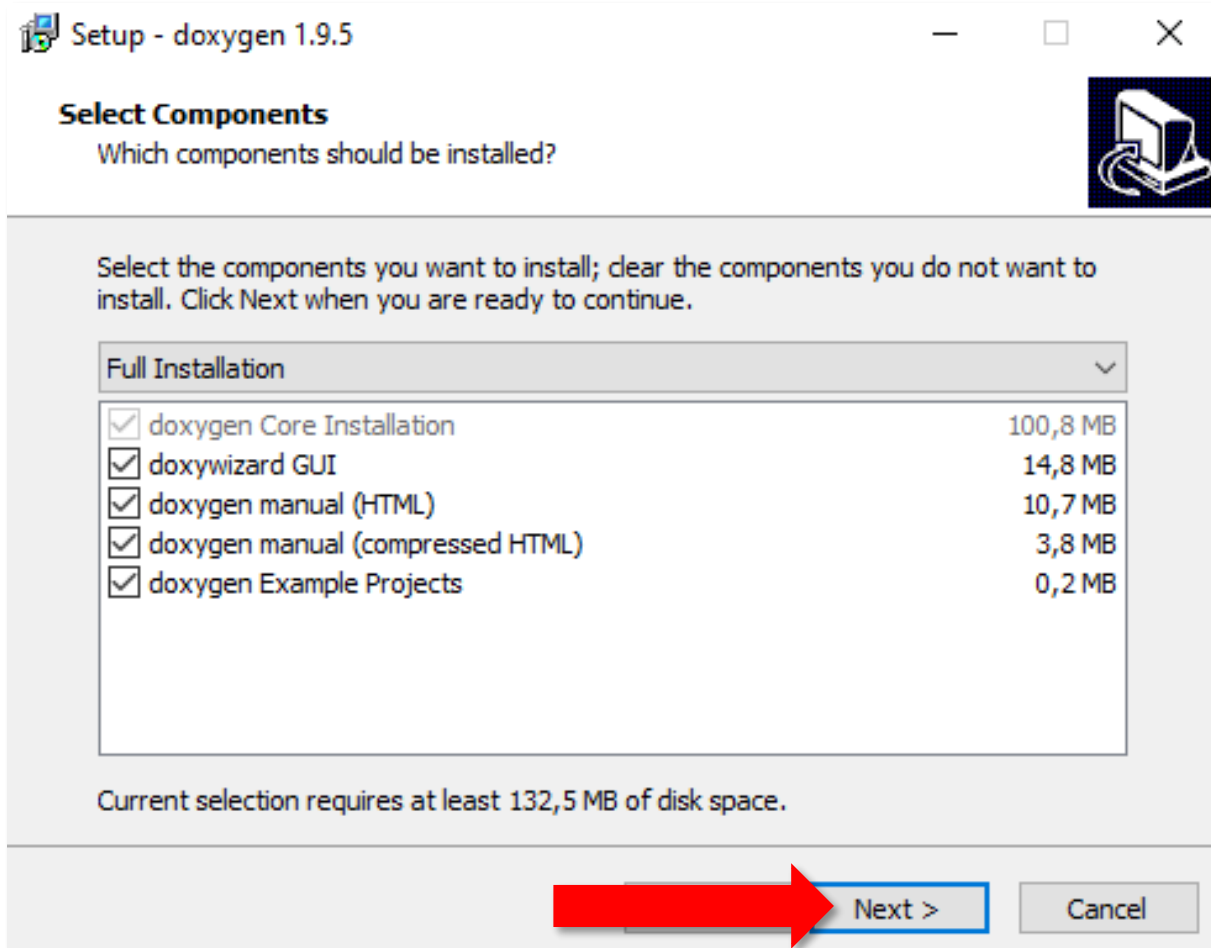
## 2.5.1. Descarga e instalación de Doxygen



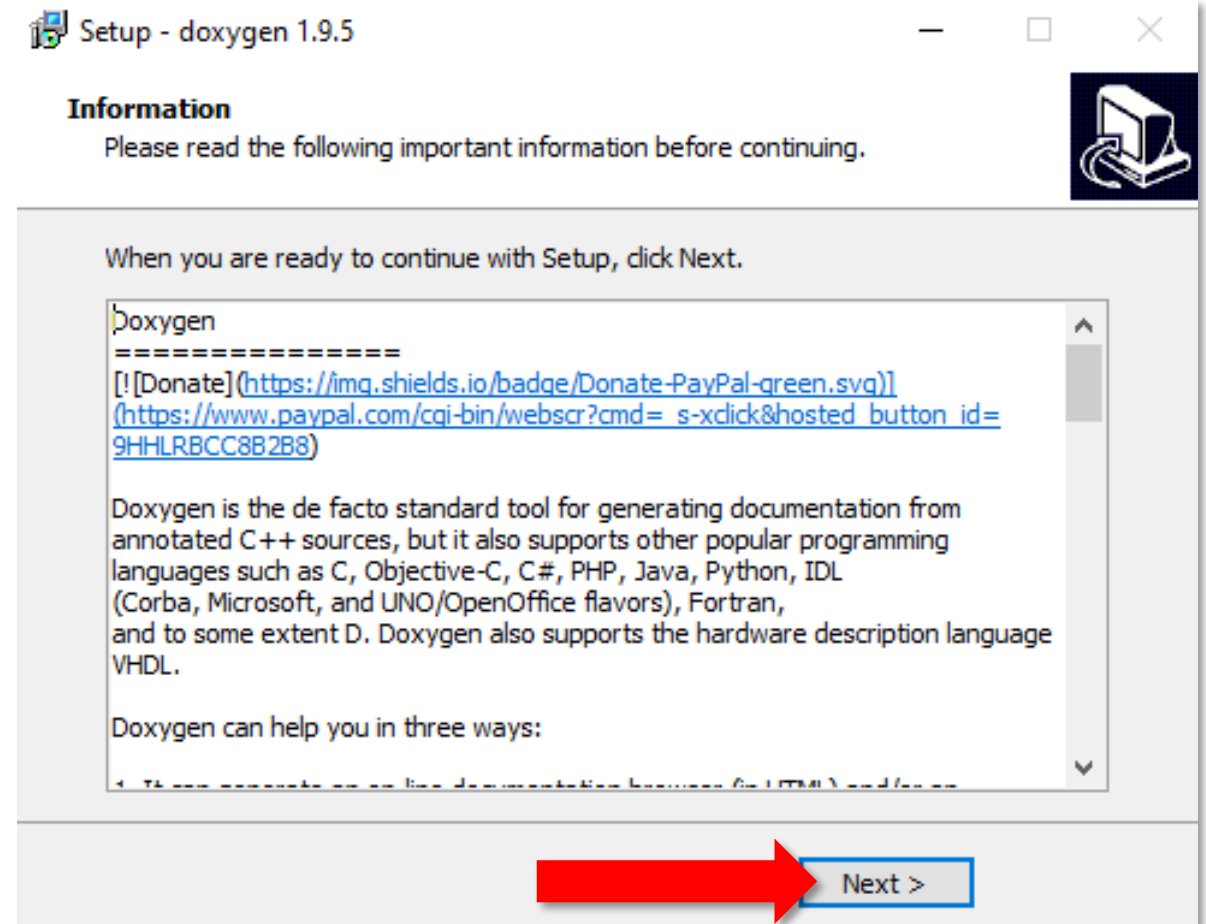
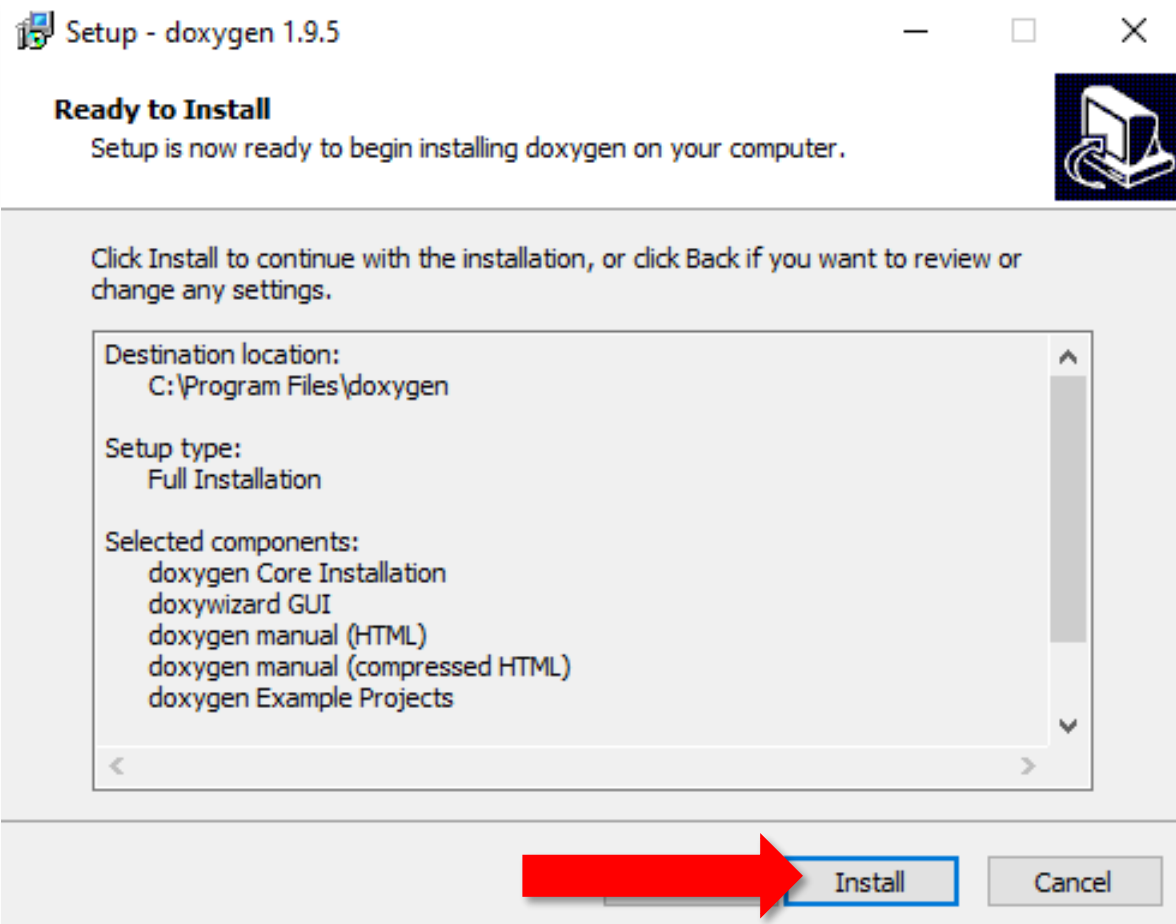
## 2.5.1. Descarga e instalación de Doxygen



## 2.5.1. Descarga e instalación de Doxygen

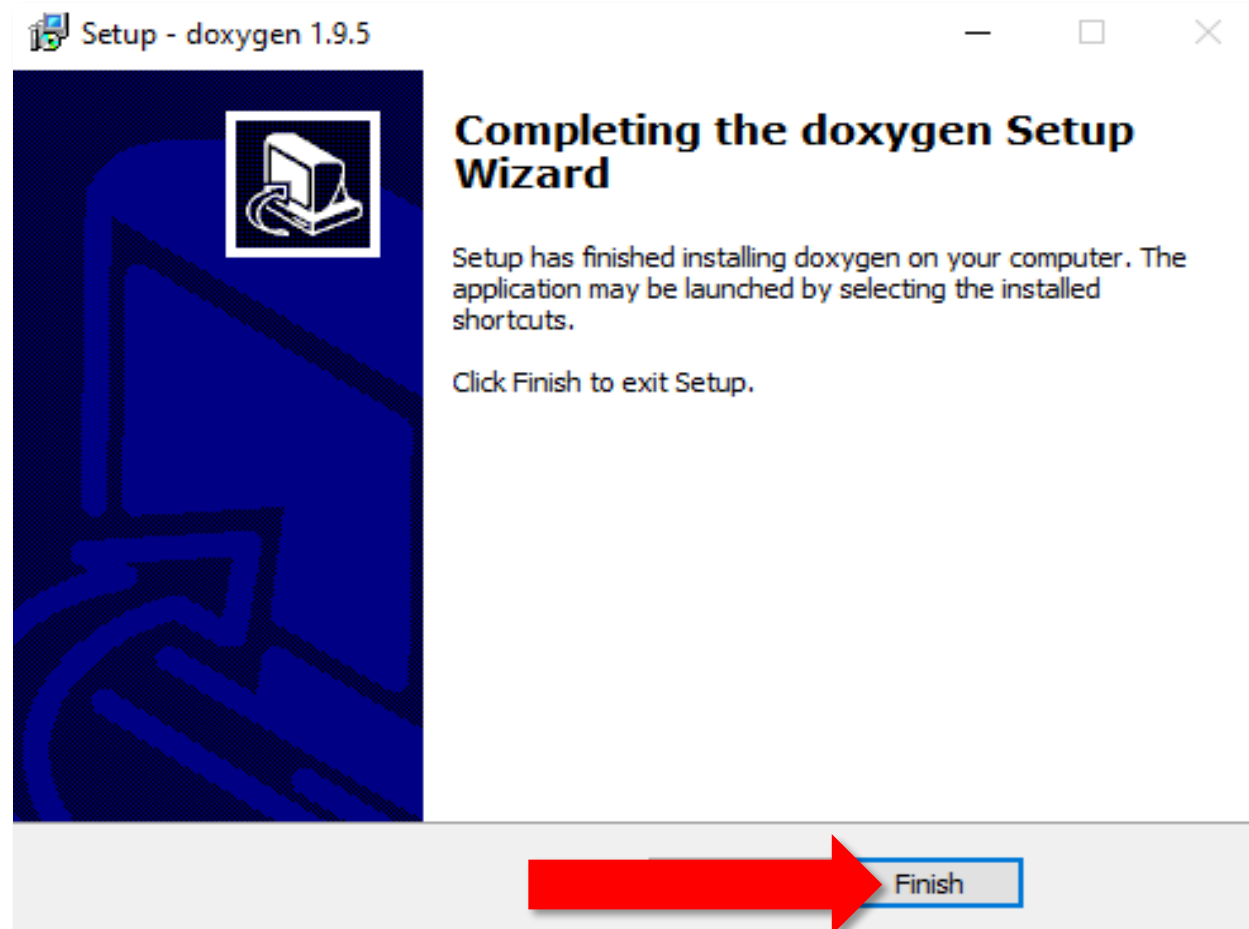


## 2.5.1. Descarga e instalación de Doxygen

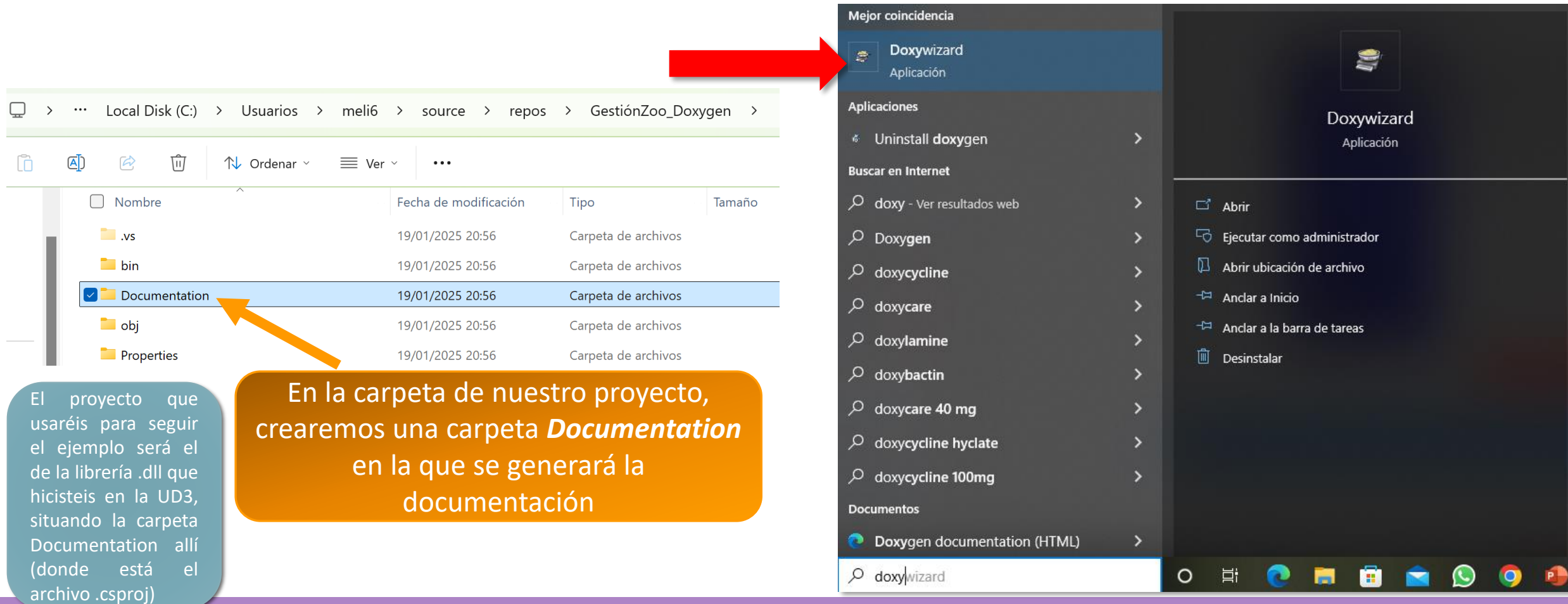


## 2.5.1. Descarga e instalación de Doxygen

---



## 2.5.2. Generación de la documentación



The image shows a Windows file explorer window on the left and a Windows Start menu search results window on the right. A red arrow points from the file explorer to the search results, and an orange arrow points from a text box to the 'Documentation' folder in the file explorer.

**File Explorer Path:** Local Disk (C:) > Usuarios > meli6 > source > repos > GestiónZoo\_Doxygen

| Nombre               | Fecha de modificación | Tipo                | Tamaño |
|----------------------|-----------------------|---------------------|--------|
| .vs                  | 19/01/2025 20:56      | Carpeta de archivos |        |
| bin                  | 19/01/2025 20:56      | Carpeta de archivos |        |
| <b>Documentation</b> | 19/01/2025 20:56      | Carpeta de archivos |        |
| obj                  | 19/01/2025 20:56      | Carpeta de archivos |        |
| Properties           | 19/01/2025 20:56      | Carpeta de archivos |        |

**Windows Start Menu Search Results:**

- Mejor coincidencia: Doxywizard (Aplicación)
- Aplicaciones: Uninstall doxygen
- Buscar en Internet: doxy - Ver resultados web, Doxygen, doxycycline, doxycare, doxylamine, doxybactin, doxycare 40 mg, doxycycline hyclate, doxycycline 100mg
- Documentos: Doxygen documentation (HTML)
- doxywizard

**Doxywizard Application Context Menu:**

- Abrir
- Ejecutar como administrador
- Abrir ubicación de archivo
- Anclar a Inicio
- Anclar a la barra de tareas
- Desinstalar

**Text Boxes:**

- El proyecto que usaréis para seguir el ejemplo será el de la librería .dll que hicisteis en la UD3, situando la carpeta Documentation allí (donde está el archivo .csproj)
- En la carpeta de nuestro proyecto, crearemos una carpeta **Documentation** en la que se generará la documentación

## 2.5.2. Generación de la documentación

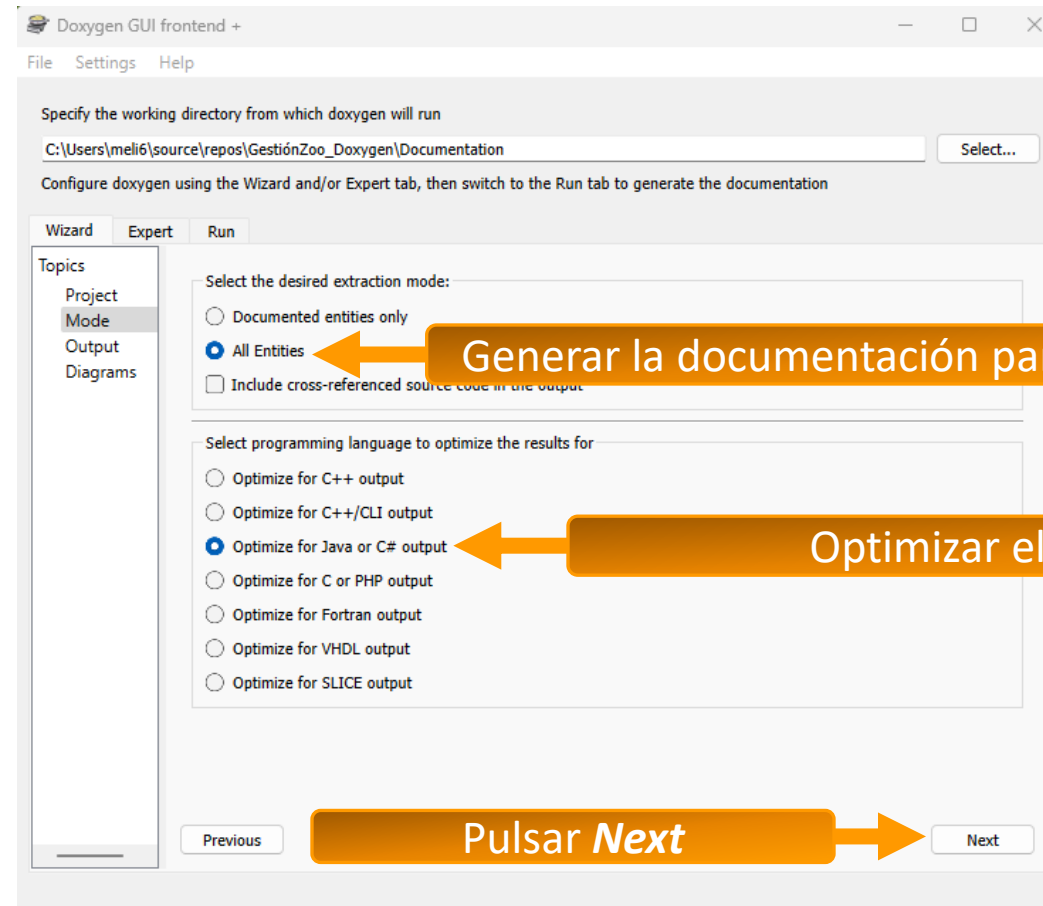
The screenshot shows the 'Doxygen GUI frontend' window with the 'Wizard' tab selected. The interface includes a 'Topics' sidebar on the left and a main configuration area on the right. Orange callout boxes with arrows point to specific fields and buttons, providing a guided tour of the setup process.

**Callouts and their corresponding fields:**

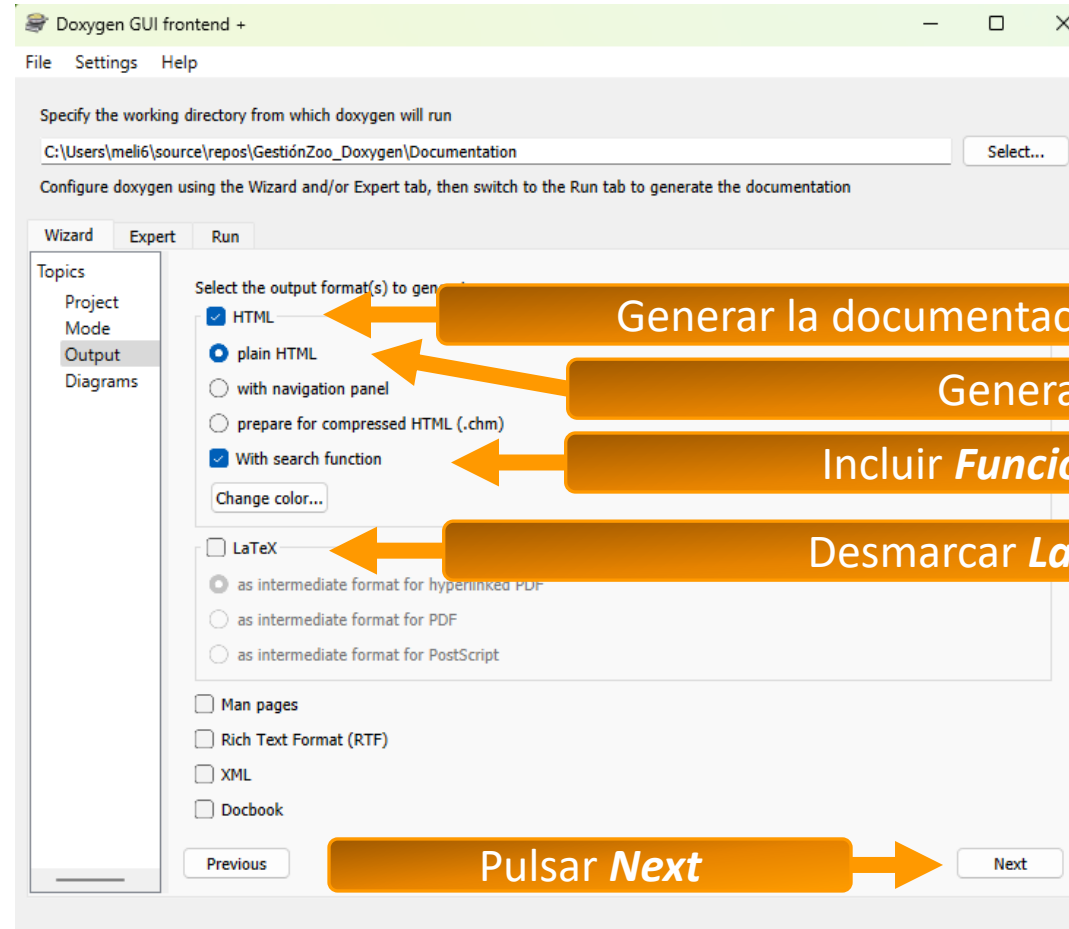
- Carpeta de trabajo** points to the 'Specify the working directory from which doxygen will run' field, which contains the path `C:\Users\meli6\source\repos\GestiónZoo_Doxygen\Documentation`.
- Resumen del proyecto** points to the 'Project' entry in the 'Topics' sidebar.
- Nombre del proyecto** points to the 'Project name' field, which contains `UD3T14CarmenCampos`.
- Versión del proyecto** points to the 'Project version or id' field, which contains `0.1`.
- Carpeta donde está el código** points to the 'Source code directory' field, which contains `i6\source\repos\GestiónZoo_Doxygen`.
- Carpeta en la que se generará la documentación** points to the 'Destination directory' field, which contains `s:\GestiónZoo_Doxygen\Documentation`.
- Pulsar *Next*** points to the 'Next' button at the bottom right.

The 'Wizard' tab also includes a 'Project synopsis' field with the text 'Primer contacto con Doxygen' and a 'Project logo' section with a 'Select...' button and the text 'No Project logo selected.'.

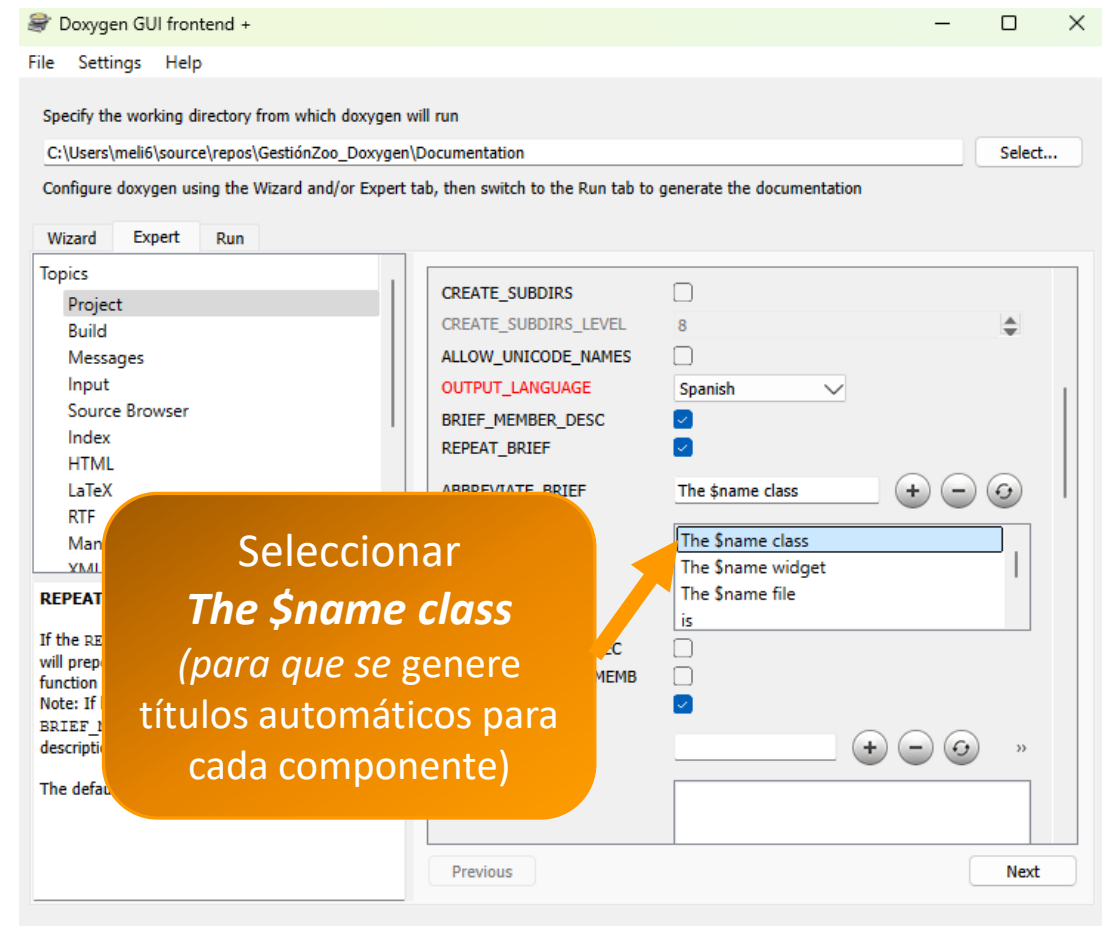
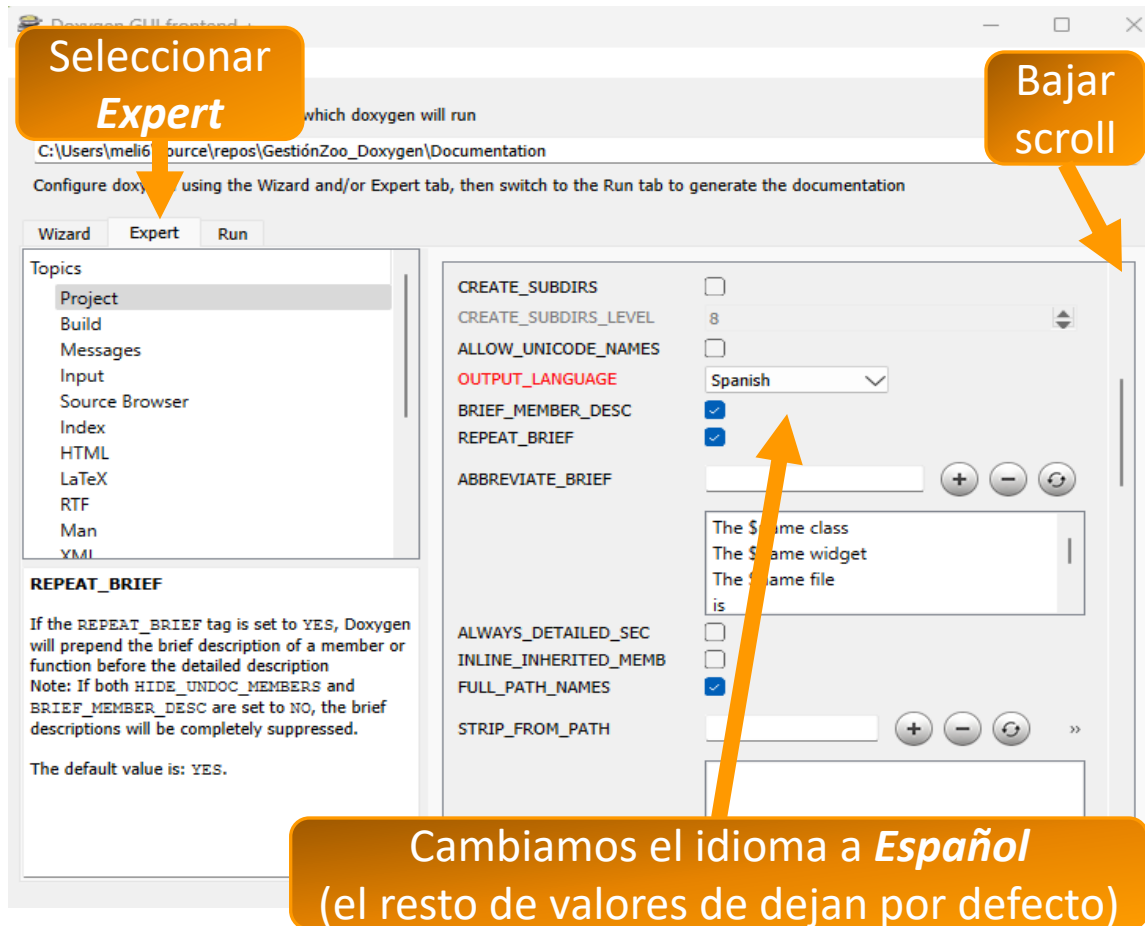
## 2.5.2. Generación de la documentación



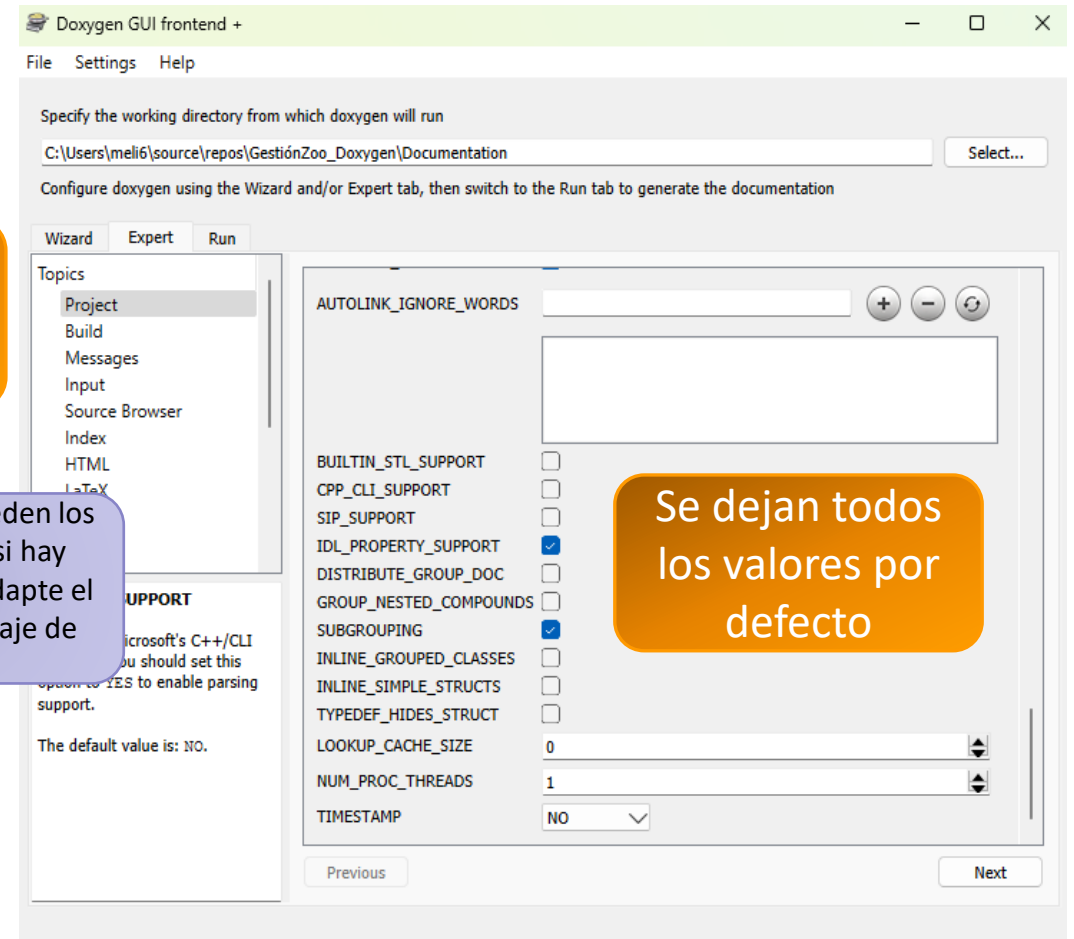
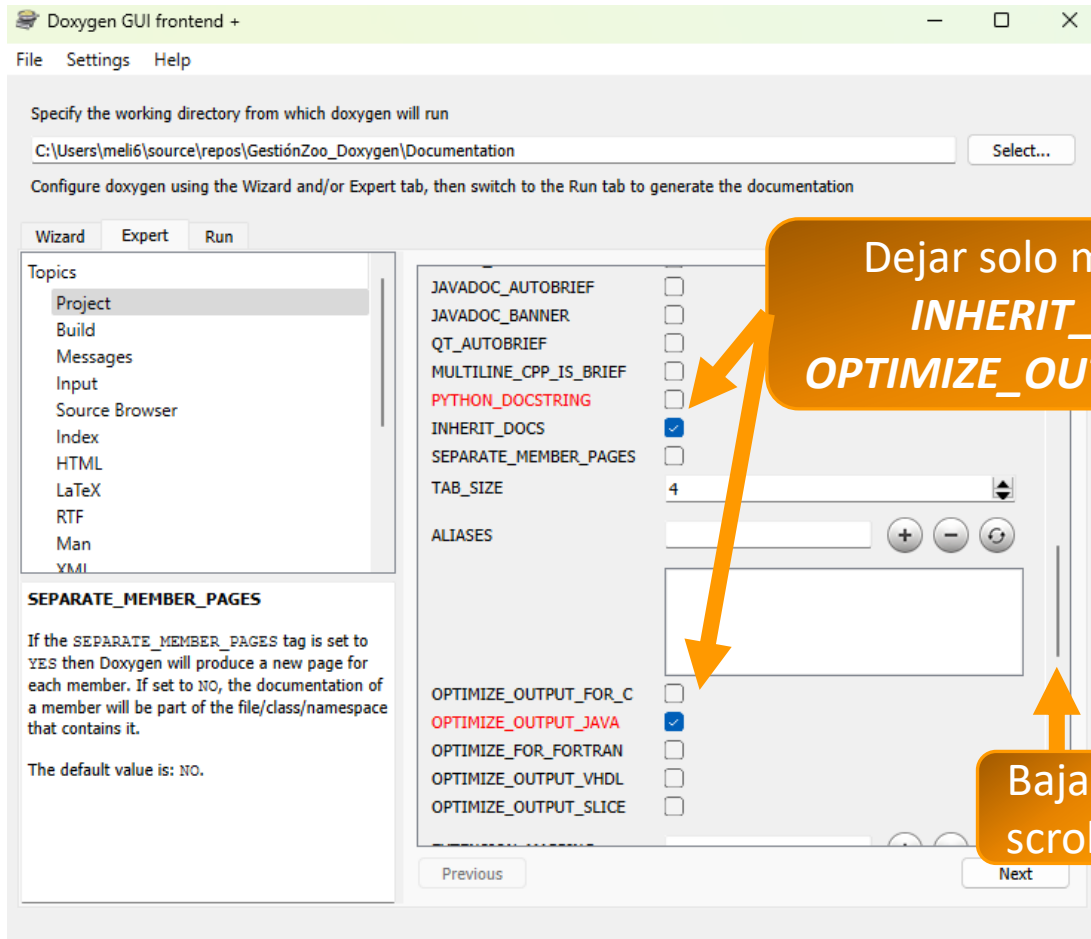
## 2.5.2. Generación de la documentación



## 2.5.2. Generación de la documentación



## 2.5.2. Generación de la documentación



## 2.5.2. Generación de la documentación

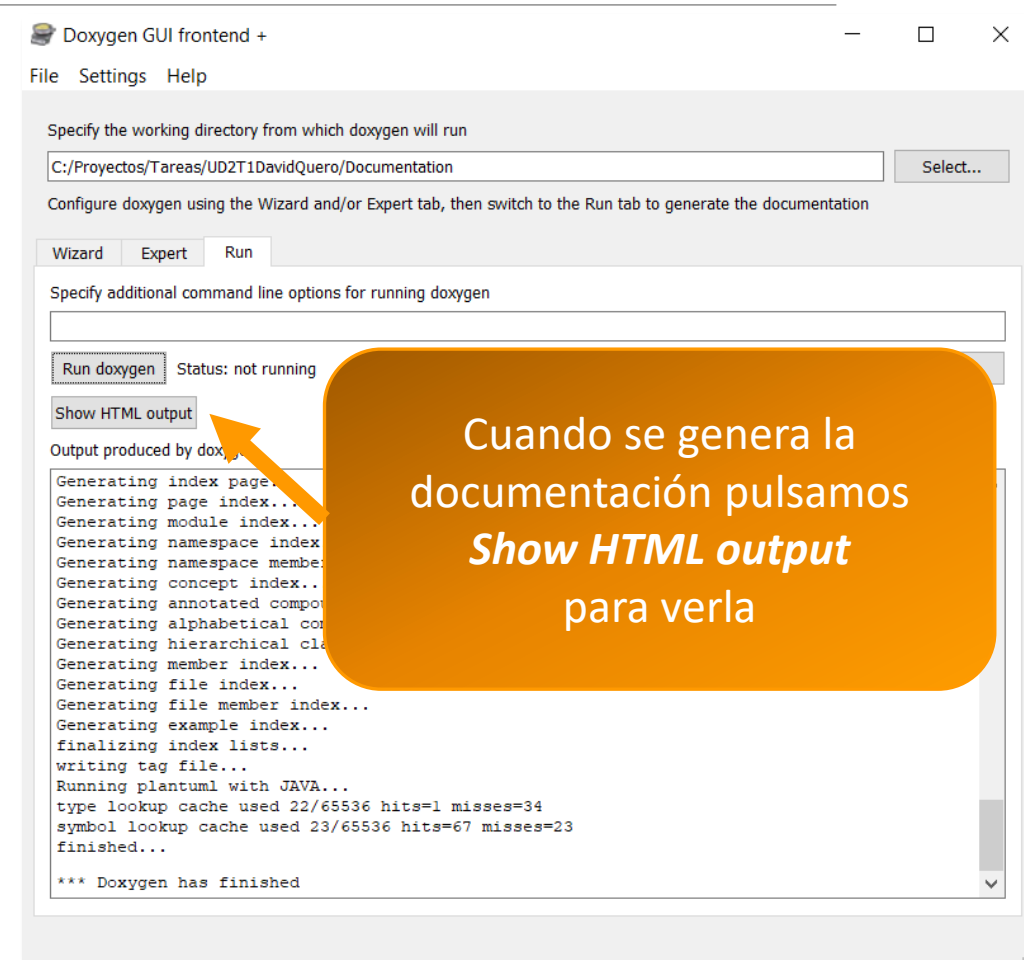
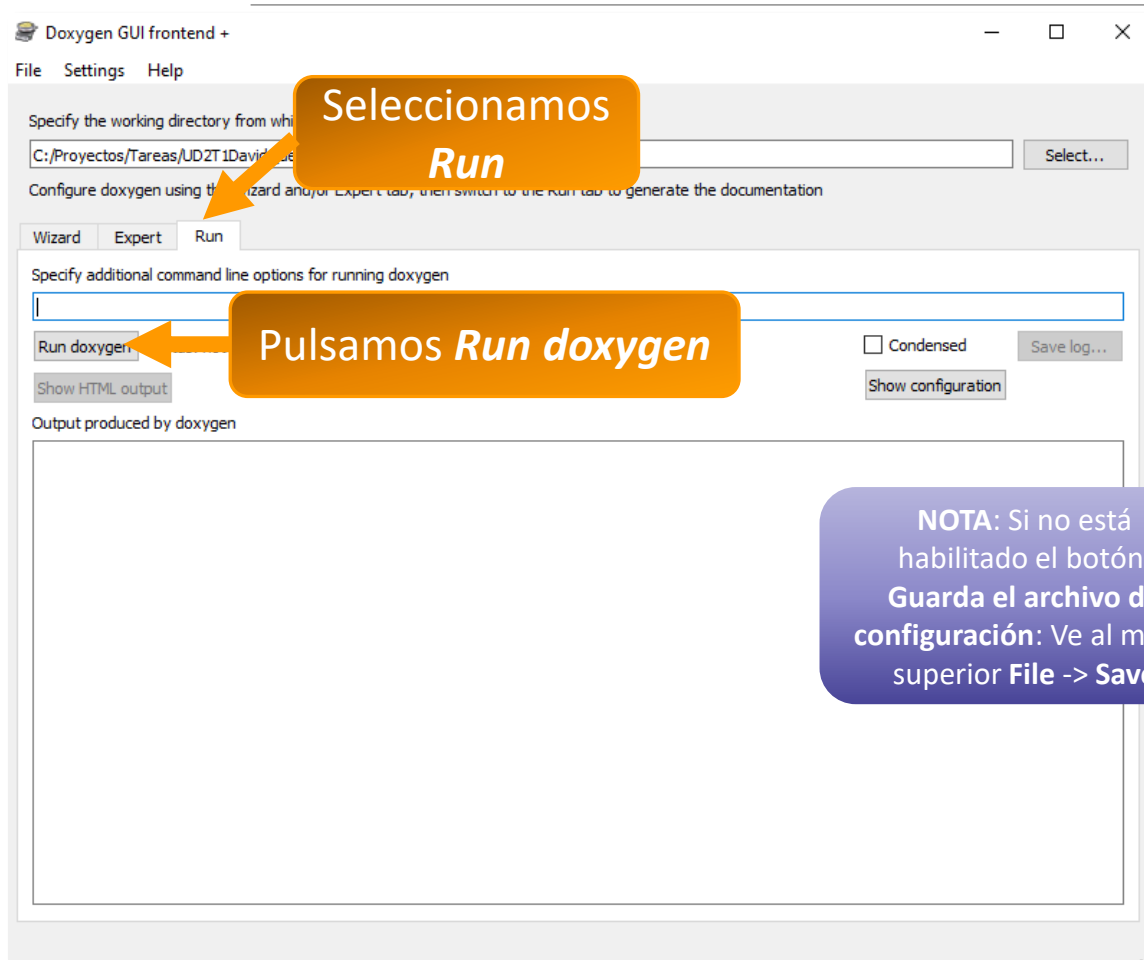
The screenshot shows the Doxygen GUI frontend window. The 'Topics' list on the left has 'Build' selected. The 'Run' tab is active, showing a list of configuration options. The 'EXTRACT\_PRIVATE' checkbox is checked, and an orange callout points to it with the text 'Dejamos los valores por defecto y nos aseguramos de tener también marcado: **EXTRACT\_PRIVATE**'. Another orange callout points to the 'Build' option in the Topics list with the text 'Seleccionamos **Build**'. A purple callout points to the 'EXTRACT\_PRIVATE' checkbox with the text 'Para que se documente los elementos privados también'.

Seleccionamos **Build**

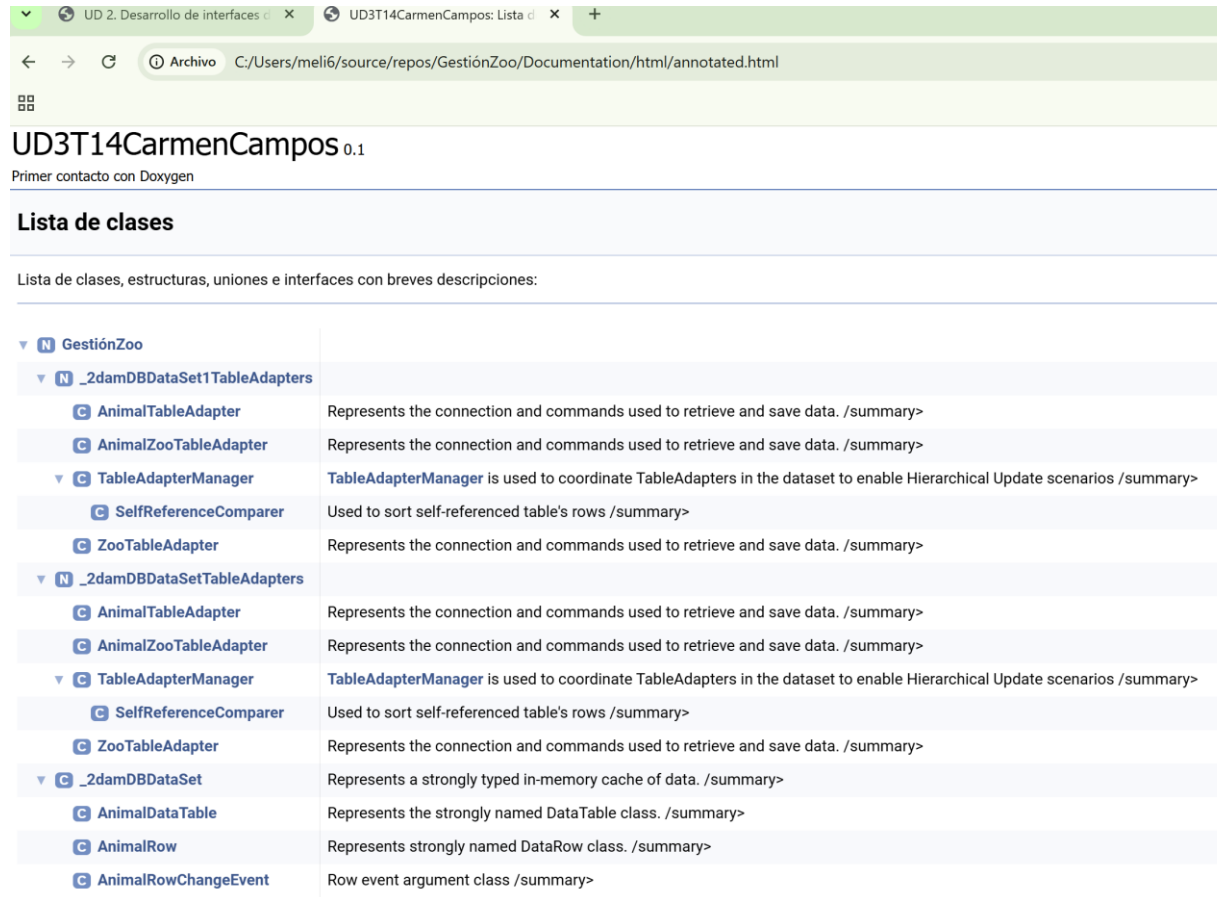
Dejamos los valores por defecto y nos aseguramos de tener también marcado: **EXTRACT\_PRIVATE**

Para que se documente los elementos privados también

## 2.5.2. Generación de la documentación



## 2.5.2. Generación de la documentación



UD 2. Desarrollo de interfaces d x UD3T14CarmenCampos: Lista d x +

← → ↻ Archivo C:/Users/meli6/source/repos/GestiónZoo/Documentation/html/annotated.html

☐☐☐

### UD3T14CarmenCampos 0.1

Primer contacto con Doxygen

#### Lista de clases

Lista de clases, estructuras, uniones e interfaces con breves descripciones:

|                                 |                                                                                                                                 |
|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| ▼ N GestiónZoo                  |                                                                                                                                 |
| ▼ N _2damDBDataSetTableAdapters |                                                                                                                                 |
| C AnimalTableAdapter            | Represents the connection and commands used to retrieve and save data. /summary>                                                |
| C AnimalZooTableAdapter         | Represents the connection and commands used to retrieve and save data. /summary>                                                |
| ▼ C TableAdapterManager         | <b>TableAdapterManager</b> is used to coordinate TableAdapters in the dataset to enable Hierarchical Update scenarios /summary> |
| C SelfReferenceComparer         | Used to sort self-referenced table's rows /summary>                                                                             |
| C ZooTableAdapter               | Represents the connection and commands used to retrieve and save data. /summary>                                                |
| ▼ N _2damDBDataSetTableAdapters |                                                                                                                                 |
| C AnimalTableAdapter            | Represents the connection and commands used to retrieve and save data. /summary>                                                |
| C AnimalZooTableAdapter         | Represents the connection and commands used to retrieve and save data. /summary>                                                |
| ▼ C TableAdapterManager         | <b>TableAdapterManager</b> is used to coordinate TableAdapters in the dataset to enable Hierarchical Update scenarios /summary> |
| C SelfReferenceComparer         | Used to sort self-referenced table's rows /summary>                                                                             |
| C ZooTableAdapter               | Represents the connection and commands used to retrieve and save data. /summary>                                                |
| ▼ C _2damDBDataSet              | Represents a strongly typed in-memory cache of data. /summary>                                                                  |
| C AnimalDataTable               | Represents the strongly named DataTable class. /summary>                                                                        |
| C AnimalRow                     | Represents strongly named DataRow class. /summary>                                                                              |
| C AnimalRowChangeEvent          | Row event argument class /summary>                                                                                              |

## Tarea 3: Documentación Interna y Ayuda en el Editor (IntelliSense).



**Objetivo:** Implementar comentarios de documentación profesional en el código fuente de la librería de controles para habilitar la ayuda contextual en Visual Studio.

- 1. Etiquetado XML:** Abre tu proyecto de **Librería de Controles**. Debes añadir comentarios de triple barra (///) a los elementos clave.
  - **<summary>:** Descripción del propósito del control o propiedad.
  - **<value>:** Especificar qué datos acepta (ej: "Valores entre 0 y 100")
  - **<remarks>:** Añadir notas técnicas (ej: "Requiere arquitectura x86 para la DB vinculada").
  - **<author>:** **Obligatorio**. Incluye tu nombre completo en la cabecera de cada clase para verificar la autoría.
- 2. Generación del Manual (Doxygen):** Configura Doxygen siguiendo las transparencias 38-42. **Personalización:** En la pestaña *Wizard*, el **Project Name** debe ser: "Librería de Controles - [Tu Nombre y Apellidos]"

# Tarea 3: Documentación Interna y Ayuda en el Editor (IntelliSense).

---

## 3. Elementos a documentar:

- **Clases:** BotonVisual y VisorProgreso.
- **Propiedades (Dependency Properties):** MensajeAccion y ProgresoValor.

## 4. Verificación de IntelliSense:

Tras comentar el código, abre el proyecto CRUD que consume esta librería. Pasa el ratón sobre las propiedades en el editor de código y verifica que aparece el cuadro de texto amarillo con la descripción que has escrito.

# Tarea 3: Documentación Interna y Ayuda en el Editor (IntelliSense)

---

**Entregable:** Debes entregar un archivo comprimido (.zip) con el siguiente contenido.

1. **Carpeta de la Librería:** El código fuente actualizado con los comentarios XML (///).
2. **Carpeta de Documentación:** La carpeta completa /html generada por Doxygen tras añadir los comentarios.
3. **Documento de Evidencias (PDF)→instrucciones:**

El documento debe servir para demostrar que comprendes cómo la documentación interna (el código) se transforma en ayuda externa (IntelliSense y HTML). Capturas **a ventana completa (donde se vea tu usuario de Windows/Visual Studio)**. No se aceptarán capturas sueltas; cada imagen debe ir acompañada de un breve texto explicativo siguiendo esta estructura:

# Tarea 3: Documentación Interna y Ayuda en el Editor (IntelliSense)

---

## 1. Sección 1: Impacto en el Entorno de Desarrollo (IntelliSense)

- **Captura de pantalla:** Muestra tu proyecto CRUD abierto, con el cursor sobre una de tus *Dependency Properties* (ej. MensajeAccion). Debe verse claramente el cuadro flotante de ayuda.
- **Explicación obligatoria:** Describe qué etiqueta XML (<summary>, <value> o <remarks>) está generando cada parte de ese texto que aparece en pantalla.
  - Ejemplo: "En la captura se observa cómo el texto 'Define el color del borde' proviene de la etiqueta <summary> que escribí en la librería".

## 2. Sección 2: Evolución del Manual Técnico (Doxygen)

- **Captura A (Versión Inicial):** Muestra la página de la clase en el HTML antes de añadir los comentarios (donde solo se ve el nombre del método o propiedad).
- **Captura B (Versión Enriquecida):** Muestra la misma sección tras ejecutar Doxygen con tus comentarios XML terminados.
- **Explicación obligatoria:** Indica al menos **dos mejoras visibles**.

Ejemplo: "Se puede observar que ahora la tabla de miembros incluye una descripción detallada y que la sección de 'Detalles' muestra las advertencias que incluí en el código".

# Tarea 3: Documentación Interna y Ayuda en el Editor (IntelliSense)

---

## 3. Sección 3: Impacto en el Entorno de Desarrollo (IntelliSense)

- **Captura de pantalla:** Muestra la carpeta de tu ordenador donde se ha generado el manual, señalando el archivo **index.html**.

# Tarea 4: Manual de Referencia Profesional y Repositorio (Markdown)

---

**Objetivo:** Crear un archivo de presentación profesional (**README.md**) que sirva de "puerta de entrada" para cualquier desarrollador que quiera usar tu librería, unificando toda la información técnica y de instalación.

## 1. Creación del archivo en Visual Studio

No utilices Word ni programas externos. Sigue estos pasos dentro de tu solución:

1. En el **Explorador de Soluciones**, haz clic derecho sobre el nombre de tu **Solución** o del **Proyecto de Librería**.
2. Selecciona **Añadir > Nuevo elemento...**
3. Busca "Archivo de texto", pero asegúrate de ponerle el nombre exacto: **README.md** (la extensión .md es vital).
4. Visual Studio reconocerá el formato. Puedes ver cómo va quedando el diseño pulsando el botón **"Vista previa" (Preview)** que aparecerá en la parte superior derecha del editor.

# Tarea 4: Manual de Referencia Profesional y Repositorio (Markdown)

---

## 2. Contenido del README.md

El archivo debe estar organizado con los siguientes apartados:

- **Ficha del Proyecto y Autor:** Título de la librería y tu nombre completo (autoría).
- **Guía de Instalación Rápida:** Pasos numerados para que otro desarrollador sepa cómo referenciar tu DLL y usarla en XAML.
- **Tabla de Referencia:** Crea una tabla Markdown con las columnas: **Control**, **Propiedad (DP)** y **Descripción**. (Usa la información técnica que ya generaste en la Tarea 3).
- **Nota Técnica Crítica:** Explica de forma destacada la necesidad de configurar la arquitectura en x86 para la compatibilidad con SQLite. También aclara cualquier punto relativo a las versiones, etc. que sea necesario para la correcta ejecución de la librería en cualquier proyecto.

## 3. Aclaración importante sobre la Sintaxis

Es fundamental que no confundas las herramientas:

**Tarea 3 (XML):** Usamos etiquetas como `<summary>` dentro del código .cs para alimentar a Doxygen e IntelliSense.

**Tarea 4 (Markdown):** Usamos símbolos como `#`, `**` o `|` en un archivo independiente .md. **Aquí no se usan etiquetas XML.** El lenguaje Markdown es mucho más limpio y visual.

# Tarea 4: Manual de Referencia Profesional y Repositorio (Markdown)

---

## 4. Recursos para aprender la sintaxis Markdown

Como este lenguaje es nuevo para ti, consulta estas guías rápidas para saber cómo hacer títulos, tablas o negritas en pocos minutos:

Tutorial Interactivo (10 min): Aprende practicando.

Guía Rápida de GitHub: El estándar que verás en el mundo laboral. 

## 5. Formato y Contenido de la Entrega

Debes entregar un archivo comprimido (**ZIP**) con el nombre Apellido\_Nombre\_Tarea4.zip que contenga:

1. **El archivo original**: El fichero README.md que has creado en Visual Studio.
2. **Capturas del "Preview" (Legibles)**: Una o varias imágenes (PNG/JPG) de la vista previa de Visual Studio.  
*Entrégalas en un único pdf.*

**Requisito**: Debe verse el contenido **completo** del documento. Si es largo, realiza varias capturas (ej. Parte superior y Parte inferior).

**Identidad**: Al menos una captura debe ser de la ventana completa de Visual Studio, donde se vea el explorador de soluciones **con el nombre del proyecto personalizado**.